Python *requests* Library

Complete Method Reference & Usage Guide · All parameters with examples · Sessions, Auth & Error handling

Install:	`pip install requests`	. **Import:**	`import requests`	. **Docs:**	`https://docs.python-requests.org`

1 · Core HTTP Methods

Method	Signature	When to Use	Example
requests.get()	`requests.get(url, params=None, **kwargs)`	**Retrieve data from a server without side effects. Use for fetching pages, querying APIs, downloading files, or any read-only operation.**	`r = requests.get( 'https://api.github.com/users/octocat', params={'per_page': 10, 'page': 2}, headers={'Accept': 'application/json'}, timeout=5 ) print(r.json()['login'])`
requests.post()	`requests.post(url, data=None, json=None, **kwargs)`	**Send data to create a new resource. Common for form submissions, login endpoints, uploading files, or triggering actions.**	`# JSON body requests.post('https://api.example.com/users', json={'name': 'Alice', 'role': 'admin'}) # Form-encoded requests.post('https://site.com/login', data={'username': 'alice', 'password': 's3cr3t'})`
requests.put()	`requests.put(url, data=None, **kwargs)`	**Replace an entire resource at a known URL. Idempotent — calling it multiple times produces the same result. Use when the client controls the full resource state.**	`requests.put( 'https://api.example.com/users/42', json={ 'name': 'Alice Smith', 'email': 'alice@example.com', 'role': 'admin' } )`
requests.patch()	`requests.patch(url, data=None, **kwargs)`	**Partially update a resource. Send only the fields that changed. More efficient than PUT when updating one or two fields on a large object.**	`requests.patch( 'https://api.example.com/users/42', json={'email': 'newemail@example.com'} )`
requests.delete()	`requests.delete(url, **kwargs)`	**Remove a resource from the server. Usually requires authentication. Some APIs use a body or query param to confirm deletion.**	`requests.delete( 'https://api.example.com/users/42', headers={'Authorization': 'Bearer TOKEN'} )`

Method	Signature	When to Use	Example
requests.head()	`requests.head(url, **kwargs)`	Like GET but returns headers only (no body). Use to check if a resource exists, get its Content-Type, size, or Last-Modified before committing to a full download.	`r = requests.head('https://example.com/large-file.zip') size = r.headers.get('Content-Length') print(f'File is {int(size)/1e6:.1f} MB')`
requests.options()	`requests.options(url, **kwargs)`	Discover which HTTP methods a server supports for a URL. Also used in CORS pre-flight checks by browsers.	`r = requests.options('https://api.example.com/users') print(r.headers.get('Allow')) # e.g. 'GET, POST, OPTIONS'`
requests.request()	`requests.request(method, url, **kwargs)`	Generic method used internally by all above helpers. Use when the HTTP verb is dynamic (e.g. determined at runtime or from config) or for non-standard verbs like PROPFIND (WebDAV).	`verb = 'PATCH' requests.request( method=verb, url='https://api.example.com/items/1', json={'status': 'active'} )`

2 · Common Request Parameters

Parameter	Type	When to Use	Example
params	`dict | list | bytes | str`	Append query string to the URL. Use for filtering, pagination, search terms, or any read-only parameter that belongs in the URL.	`requests.get('https://api.example.com/search', params={'q': 'python', 'sort': 'stars', 'page': 3}) # URL → …/search?q=python&sort;=stars&page;=3 # Multi-value key requests.get(url, params=[('tag', 'web'), ('tag', 'api')])`
data	`dict | list | bytes | file-obj`	Send URL-encoded form data (application/x-www-form-urlencoded) or raw bytes. Use for HTML form submissions, legacy APIs, or binary payloads.	`# Form requests.post(url, data={'key': 'value'}) # Raw bytes requests.post(url, data=b'\x00\x01binary') # Streaming file with open('report.csv') as f: requests.post(url, data=f)`
json	`any JSON-serializable`	Serialize a Python object to JSON and set Content-Type to application/json automatically. Preferred over data= when talking to modern REST APIs.	`requests.post('https://api.example.com/events', json={ 'type': 'purchase', 'items': [{'id': 99, 'qty': 2}], 'total': 29.99 } )`
headers	`dict`	Override or add HTTP request headers: auth tokens, custom content types, accept formats, user-agent strings, or API versioning.	`requests.get(url, headers={ 'Authorization': 'Bearer eyJhbGci...', 'Accept': 'application/vnd.api+json', 'X-Request-ID': 'abc-123', 'User-Agent': 'MyApp/2.0' })`
auth	`tuple | AuthBase`	Attach authentication credentials. Pass a (user, pass) tuple for HTTP Basic Auth, or use a custom AuthBase subclass for OAuth, Digest, etc.	`# Basic Auth requests.get(url, auth=('user', 'pass')) # Digest Auth from requests.auth import HTTPDigestAuth requests.get(url, auth=HTTPDigestAuth('u', 'p')) # OAuth1 (via requests-oauthlib) from requests_oauthlib import OAuth1 requests.get(url, auth=OAuth1(ck, cs, at, ats))`
timeout	`float | (float, float)`	Prevent hanging forever. Always set in production. A single float sets both connect + read timeout. A tuple sets them independently.	`# 5 s total requests.get(url, timeout=5) # 3 s to connect, 10 s to read response requests.get(url, timeout=(3, 10)) # Handle it try: r = requests.get(url, timeout=5) except requests.Timeout: print('Timed out!')`
cookies	`dict | CookieJar`	Send cookies with the request without relying on a Session. Useful for quick one-off authenticated requests or scraping session-based sites.	`requests.get(url, cookies={'session_id': 'abc123', 'csrf': 'tok'}) # Or use a CookieJar from a previous response jar = r.cookies requests.get(url2, cookies=jar)`
verify	`bool | str`	Control SSL certificate verification. Default True — never disable in production. Pass a CA bundle path to trust a private/self-signed cert.	`# Trust custom CA requests.get(url, verify='/etc/ssl/my-ca.pem') # Disable (ONLY for local dev/test) import urllib3 urllib3.disable_warnings() requests.get(url, verify=False)`
cert	`str | (str, str)`	Send a client-side TLS certificate for mutual TLS (mTLS) authentication. Required by APIs that authenticate via certificate instead of tokens.	`# Combined PEM file requests.get(url, cert='/path/client.pem') # Separate cert + key requests.get(url, cert=('/path/client.crt', '/path/client.key'))`

Parameter	Type	When to Use	Example
proxies	dict	Route requests through an HTTP/HTTPS/SOCKS proxy. Use for corporate networks, anonymization, geo-restricted content, or traffic capture.	`requests.get(url, proxies={ 'http': 'http://proxy.corp.com:8080', 'https': 'http://proxy.corp.com:8080', 'https': 'socks5://user:pass@host:1080' })`
stream	bool	When True, the response body is not immediately downloaded. Ideal for large files, real-time event streams, or when you need to inspect headers before deciding whether to download.	`r = requests.get('https://files.example.com/big.iso', stream=True, timeout=10) r.raise_for_status() with open('big.iso', 'wb') as f: for chunk in r.iter_content(chunk_size=8192): f.write(chunk)`
allow_redirects	bool	Control whether the library follows HTTP 3xx redirects. Default True for GET/HEAD, False for POST/PUT/PATCH. Set False to capture the redirect URL itself.	`r = requests.get(url, allow_redirects=False) if r.status_code in (301, 302): print('Redirects to:', r.headers['Location'])`
files	dict	Upload files as multipart/form-data. The dict maps field names to file objects or tuples of (filename, file-obj, content-type). Mix with data= to include other form fields alongside the file.	`with open('photo.jpg', 'rb') as img: requests.post(url, files={ 'avatar': ('photo.jpg', img, 'image/jpeg'), 'resume': open('cv.pdf', 'rb') }, data={'user_id': 42})`

3 · Session Object

Method / Attribute	Purpose	When to Use	Example
Session()	Creates a persistent session that reuses the same TCP connection, cookies, and default headers across multiple requests.	**Any time you make more than one request to the same host. Dramatically reduces latency and resource usage vs creating a new connection each time.**	`s = requests.Session() s.headers['Authorization'] = 'Bearer TOKEN' s.headers['Accept'] = 'application/json' for page in range(1, 6): r = s.get(url, params={'page': page}) process(r.json())`
session.get/post/…()	Same semantics as module-level methods but inherit the session's persistent state (headers, auth, cookies, proxies).	**Use inside a Session context for all actual requests.**	`with requests.Session() as s: s.auth = ('user', 'pass') r1 = s.get('https://api.example.com/login') r2 = s.get('https://api.example.com/data') # cookies from r1 auto-sent in r2`
session.headers	A dict of headers sent with every request in the session. Per-request headers are merged on top.	**Set auth tokens, user-agent, content-type defaults once instead of repeating them on every call.**	`s = requests.Session() s.headers.update({ 'User-Agent': 'MyCrawler/1.0', 'X-API-Key': os.environ['API_KEY'] })`
session.cookies	A RequestsCookieJar that persists cookies across requests, just like a browser.	**Useful when authenticating via login forms that set session cookies, or when scraping sites that require cookie continuity.**	`s = requests.Session() s.post('https://site.com/login', data={'user': 'me', 'pass': 'pw'}) # Session cookie kept automatically r = s.get('https://site.com/dashboard')`
session.auth	Default authentication applied to every request in the session.	**Set once when the entire session talks to the same authenticated API endpoint.**	`s = requests.Session() s.auth = HTTPDigestAuth('user', 'secret') s.get('https://protected.example.com/data')`
session.mount()	Attach a custom Transport Adapter to URLs matching a given prefix. Adapters control low-level connection behaviour.	**Use to configure retry logic, connection pooling, custom SSL, or to mock HTTP responses in tests.**	`from requests.adapters import HTTPAdapter from urllib3.util.retry import Retry retry = Retry(total=3, backoff_factor=0.5, status_forcelist=[500,502,503]) s = requests.Session() s.mount('https://', HTTPAdapter(max_retries=retry))`
session.close()	Release all underlying network resources and connection pools.	**Always call when you are finished with a session. Use the context manager (with statement) to have it called automatically.**	`# Preferred: context manager with requests.Session() as s: r = s.get(url) # Manual s = requests.Session() try: r = s.get(url) finally: s.close()`

4 · Response Object

Attribute / Method	Returns	When to Use	Example
r.status_code	int	Check whether the request succeeded. 2xx = success, 4xx = client error, 5xx = server error.	r = requests.get(url) if r.status_code == 200: handle(r) elif r.status_code == 404: print('Not found') elif r.status_code == 429: time.sleep(int(r.headers.get('Retry-After', 5)))
r.raise_for_status()	None or raises	Raise an HTTPError for any 4xx/5xx response. The cleanest way to handle errors without manually checking the status code every time.	r = requests.get(url) try: r.raise_for_status() except requests.HTTPError as e: print(f'HTTP {r.status_code}: {e}')
r.json()	dict / list	Parse the response body as JSON. Use for any REST API that returns application/json. Raises ValueError if the body is not valid JSON.	r = requests.get('https://api.github.com/repos/psf/requests') data = r.json() print(data['stargazers_count'], data['language'])
r.text	str	Get the response body decoded as a Unicode string (uses r.encoding). Use for HTML scraping, plain-text APIs, or XML responses.	r = requests.get('https://example.com') if '' in r.text: title = r.text.split('')[1].split('')[0] print(title)
r.content	bytes	Raw bytes of the response body. Use when downloading binary files (images, PDFs, ZIPs) that must not be decoded as text.	r = requests.get('https://example.com/logo.png') with open('logo.png', 'wb') as f: f.write(r.content)
r.headers	CaseInsensitiveDict	Inspect server response headers: content type, caching policy, rate-limit remaining, redirect location, etc.	r = requests.get(url) print(r.headers['Content-Type']) print(r.headers.get('X-RateLimit-Remaining', 'N/A')) print(r.headers.get('Cache-Control'))
r.cookies	RequestsCookieJar	Access cookies set by the server. Use when you need to inspect, store, or forward specific cookies.	r = requests.post('https://site.com/login', data=creds) session_id = r.cookies.get('PHPSESSID') print('Session cookie:', session_id)
r.url	str	The final URL after redirects. Useful for logging, debugging, or when you need to know where the request actually landed.	r = requests.get('http://example.com') # HTTP → HTTPS redirect print(r.url) # https://example.com/
r.elapsed	timedelta	Total time the request took. Use for performance monitoring, logging slow endpoints, or debugging latency issues.	r = requests.get(url) print(f'Request took {r.elapsed.total_seconds():.3f}s')
r.history	list[Response]	List of redirect responses that led to the final response. Empty if no redirects occurred. Useful for auditing redirect chains.	r = requests.get('http://github.com') for redirect in r.history: print(f'{redirect.status_code} → {redirect.url}') print('Final:', r.url)
r.encoding	str | None	Encoding detected from Content-Type header. Override before accessing r.text when the server sends the wrong charset.	r = requests.get('https://legacy-site.com') r.encoding = 'windows-1252' # override auto-detect print(r.text)

Attribute / Method	Returns	When to Use	Example
r.iter_content()	generator of bytes	Iterate over the response in chunks. Use with stream=True to download large files without loading everything into memory at once.	r = requests.get(url, stream=True) r.raise_for_status() with open('file.bin', 'wb') as f: for chunk in r.iter_content(chunk_size=65536): if chunk: # skip keep-alive chunks f.write(chunk)
r.iter_lines()	generator of str/bytes	Stream a response line-by-line. Perfect for Server-Sent Events (SSE), newline-delimited JSON (NDJSON), or large log files.	r = requests.get('https://api.example.com/stream', stream=True) for line in r.iter_lines(decode_unicode=True): if line: event = json.loads(line) handle_event(event)
r.ok	bool	True if status_code < 400. A quick boolean check when you want a simple pass/fail without raising an exception.	r = requests.get(url) if r.ok: return r.json() else: log_error(r.status_code, r.text) return None
r.links	dict	Parsed Link headers — used by APIs (GitHub, GitLab) for pagination. Returns dict with 'next', 'prev', 'last', 'first' keys.	url = 'https://api.github.com/users' while url: r = requests.get(url) process(r.json()) url = r.links.get('next', {}).get('url')

5 · Exceptions

Exception	Inherits From	When It Fires	Handling Example
RequestException	IOError	Base class for all requests exceptions. Catching this handles every network or HTTP error in one clause.	try: r = requests.get(url, timeout=5) r.raise_for_status() except requests.RequestException as e: logger.error('Request failed: %s', e)
HTTPError	RequestException	Raised by raise_for_status() when the server returns a 4xx or 5xx status code.	try: r.raise_for_status() except requests.HTTPError as e: if e.response.status_code == 401: refresh_token()
ConnectionError	RequestException	Network-level problem: DNS failure, refused connection, network unreachable, etc. The server was never reached.	try: r = requests.get(url) except requests.ConnectionError: print('Could not reach server — check DNS/network')
Timeout	ConnectionError	Fired when the request exceeds the timeout (connect or read). Always set timeout= and catch this in production code.	try: r = requests.get(url, timeout=3) except requests.Timeout: print('Request timed out — retry or fail fast')
TooManyRedirects	RequestException	More than max_redirects (default 30) redirects were followed. Usually indicates a redirect loop.	try: r = requests.get(url, max_redirects=5) except requests.TooManyRedirects: print('Redirect loop detected at:', url)
SSLError	ConnectionError	SSL certificate verification failed or TLS handshake error. Check the certificate chain or supply a custom CA bundle.	try: r = requests.get(url) except requests.exceptions.SSLError: # In prod: fix the cert. Never disable verify. r = requests.get(url, verify='/path/to/ca-bundle.crt')
ProxyError	ConnectionError	Failure communicating with the configured proxy.	try: r = requests.get(url, proxies=proxies) except requests.exceptions.ProxyError: print('Proxy unreachable — check proxy settings')
ChunkedEncodingError	RequestException	The server declared chunked encoding but the stream was corrupted or cut off mid-transfer. Common during large downloads.	try: for chunk in r.iter_content(8192): f.write(chunk) except requests.ChunkedEncodingError: print('Stream interrupted — partial file')

6 · Authentication Helpers

Class / Helper	Auth Type	When to Use	Example
HTTPBasicAuth	Basic Auth (Base64)	Simple username/password over HTTPS. Supported by most HTTP servers and many REST APIs. Always use over TLS — credentials are only base64-encoded.	from requests.auth import HTTPBasicAuth requests.get(url, auth=HTTPBasicAuth('user', 'password')) # Shorthand tuple also works: requests.get(url, auth=('user', 'password'))

Class / Helper	Auth Type	When to Use	Example
HTTPDigestAuth	Digest Auth (MD5 hash)	**Challenge-response scheme that avoids sending the password in clear text. Found on older routers, WebDAV servers, and some enterprise systems.**	`from requests.auth import HTTPDigestAuth requests.get( 'http://router.local/api', auth=HTTPDigestAuth('admin', 'admin') )`
BearerAuth (custom)	Bearer Token (OAuth 2)	**Modern OAuth2 / JWT APIs. Pass the token in the Authorization header. You can use a header dict or write a tiny AuthBase subclass.**	`# Via header requests.get(url, headers={'Authorization': f'Bearer {token}'}) # Reusable subclass class BearerAuth(requests.auth.AuthBase): def __init__(self, token): self.token = token def __call__(self, r): r.headers['Authorization'] = f'Bearer {self.token}' return r requests.get(url, auth=BearerAuth(token))`
requests_oauthlib OAuth1	OAuth 1.0a	**Twitter, Flickr, and legacy APIs that use OAuth 1.0a request signing. Requires consumer key/secret and access token/secret.**	`from requests_oauthlib import OAuth1 auth = OAuth1( consumer_key, consumer_secret, access_token, access_token_secret ) requests.get('https://api.twitter.com/1.1/…', auth=auth)`
requests_oauthlib OAuth2Session	OAuth 2.0	**APIs using the full OAuth 2.0 flow (authorization code, client credentials, token refresh). Handles token fetching and refresh automatically.**	`from requests_oauthlib import OAuth2Session oauth = OAuth2Session(client_id, redirect_uri='https://app.example.com/cb') authorization_url, state = oauth.authorization_url('https://provider.example.com/oauth/authorize') # … user visits URL, you get code … token = oauth.fetch_token('https://provider.example.com/oauth/token', authorization_response=callback_url, client_secret=client_secret) oauth.get('https://api.example.com/me')`

■ Pro tip: Always use a Session for multiple requests, always set timeout=, always call r.raise_for_status(), and never disable verify= in production.